

DOCUMENTACIÓN TÉCNICA

MéridaActiva

Manual Técnico

MAYO
2026

· REACT · TYPESCRIPT ·
· SUPABASE · VITE

· MERIDAACTIVA.VERCEL.APP

Lucía Garrido Chocarro · DAW B

ÍNDICE DE CONTENIDOS

01	Arquitectura general	02	Estructura de directorios
03	Tecnologías y dependencias	04	Variables de entorno
05	Base de datos — Supabase	06	Sistema de autenticación
07	Rutas y navegación	08	Módulo de IA — Rutas y Chat
09	API Serverless	10	Componentes principales
11	Hooks personalizados	12	PWA y Service Worker
13	Sistema de tipos TypeScript	14	SEO y metadatos
15	Consideraciones de rendimiento		

01

Arquitectura general

MéridaActiva es una **SPA (Single Page Application)** con arquitectura cliente-servidor desacoplada. El frontend se comunica con **Supabase** directamente para CRUD de datos. Para las funciones de IA (chat y rutas), el frontend llama a las **API Serverless** de Vercel, que actúan como proxy hacia **OpenRouter** para no exponer la API key en cliente. La autenticación es gestionada íntegramente por **Supabase Auth** con JWT.

CAPA	TECNOLOGÍA	RESPONSABILIDAD
Frontend	React 19 + TypeScript + Vite	UI, enrutamiento, estado
Backend / Auth	Supabase (PostgreSQL + RLS)	CRUD, autenticación JWT, storage
Serverless	Vercel Functions (/api/)	Proxy IA, email, eliminación de cuenta
IA	OpenRouter (Gemini 2.5 / GPT-4o-mini)	Generación de rutas y chat
Rate limiting	Upstash Redis	Límites de peticiones por IP



02

Estructura de directorios

El repositorio sigue una estructura modular por dominio:

DIRECTORIO / ARCHIVO	DESCRIPCIÓN
api/	Serverless Functions de Vercel (chat, generar-ruta, contacto, eliminar-cuenta)
public/	Assets estáticos: imágenes, iconos PWA, robots.txt, og-default.png

DIRECTORIO / ARCHIVO	DESCRIPCIÓN
src/App.tsx	Componente raíz con providers globales
src/Routes.tsx	Definición de rutas con React Router v6
src/types.ts	Interfaces TypeScript centralizadas
src/componentes/	Componentes reutilizables (Navbar, Mapa, LazyImg, animaciones...)
src/context/	AuthContext — estado global de autenticación
src/hooks/	useFavoritos, useOfflineStatus, useSeoMeta
src/paginas/ publicas/	Páginas accesibles sin login
src/paginas/ privadas/	Páginas que requieren login
src/paginas/admin/	Panel de administración (rol Admin/Gestor)
src/utils/	geminiService.ts, perfilUsuario.ts, toast.ts
vite.config.ts	Configuración de Vite + PWA
vercel.json	Configuración de despliegue en Vercel



03 Tecnologías y dependencias

Dependencias de producción

PAQUETE	VERSIÓN	USO
react	^19.2.0	Framework UI
react-router-dom	^6.30.3	Enrutamiento SPA
@supabase/supabase-js	^2.93.1	Cliente Supabase (Auth + DB)

PAQUETE	VERSIÓN	USO
gsap	^3.14.2	Animaciones avanzadas
leaflet + react-leaflet	^1.9.4 / ^5.0.0	Mapas interactivos
jspdf	^4.2.0	Generación de PDF
react-markdown	^10.1.0	Renderizado Markdown (Chat)
vite-plugin-pwa	^1.2.0	Service Worker / PWA
@upstash/ratelimit	^2.0.8	Rate limiting serverless
react-quill	^2.0.0	Editor de texto enriquecido
react-hot-toast	^2.6.0	Notificaciones toast
react-helmet-async	^3.0.0	Gestión de <head>
@upstash/redis	^1.37.0	Redis para rate limit
@vercel/analytics	^2.0.1	Analytics de Vercel

Dependencias de desarrollo

PAQUETE	USO
vite ^7.3.1	Bundler y dev server
@vitejs/plugin-react-swc	Compilación React con SWC
typescript ~5.9.3	Tipado estático
tailwindcss ^4.2.0	CSS utilitario
eslint + plugins	Linting
concurrently	Lanzar frontend + API en paralelo



04

Variables de entorno

Crea un archivo `.env` en la raíz del proyecto. Las variables con prefijo `VITE_` son accesibles en el frontend mediante `import.meta.env.VITE_*`. Las demás solo están disponibles en las Serverless Functions.

VARIABLE	ÁMBITO	DESCRIPCIÓN
VITE_SUPABASE_URL	Frontend	URL del proyecto Supabase
VITE_SUPABASE_ANON_KEY	Frontend	Clave anónima de Supabase
API_KEY_IA	Servidor	API key de OpenRouter
OPENROUTER_MODEL	Servidor	Modelo IA (opcional, hay fallbacks)
UPSTASH_REDIS_REST_URL	Servidor	URL de Upstash Redis para rate limiting
UPSTASH_REDIS_REST_TOKEN	Servidor	Token de Upstash Redis
ALLOWED_ORIGIN	Servidor	Origen permitido CORS
VITE_API_BASE_URL	Frontend	Vacío en dev y producción (rutas relativas / api/)



05

Base de datos — Supabase

Tablas principales

TABLA	COLUMNAS CLAVE	NOTAS
usuarios	id (uuid), nombre, email, avatar_url, bio, rol_id	id igual que auth.users
roles	id (serial), nombre	'Usuario', 'Gestor (Editor)', 'Administrador'
eventos		Lectura pública

TABLA	COLUMNAS CLAVE	NOTAS
	id, titulo, descripcion, fecha, hora, lugar, lat/lng, imagen_url, categoria, precio, animales_permitidos, enlace_externo	
comentarios	id, evento_id, usuario_id, texto, puntuacion (1-5)	Inserción solo autenticado
favoritos	id, usuario_id, elemento_id, tipo_elemento	Solo propietario
agenda_personal	id, usuario_id, titulo, fecha, hora, nota, color	Solo propietario

Row Level Security (RLS)

Todas las tablas tienen RLS activado. Las políticas clave son: **usuarios** con lectura pública y escritura solo del propio usuario; **comentarios** con lectura pública, inserción solo autenticado y borrado solo admin o autor; **favoritos** y **agenda_personal** con acceso completo solo al propietario; **eventos** con lectura pública y escritura solo para rol Administrador o Gestor.



06 Sistema de autenticación

La autenticación está centralizada en `src/context/AuthContext.tsx` mediante el patrón **Context + Provider**. Expone: `session` (JWT de Supabase), `profile` (datos del usuario), `loading`, `signOut`, `refreshProfile` y `forceNuclearLogout`.

Ciclo de vida de la sesión

PASO	DESCRIPCIÓN
1. Inicialización	<code>getSession()</code> → valida con <code>getUser()</code> (anti-JWT-corrupto) → carga perfil
2. Listener	<code>onAuthStateChange</code> escucha <code>SIGNED_IN</code> , <code>SIGNED_OUT</code> , <code>TOKEN_REFRESHED</code> de forma asíncrona

PASO	DESCRIPCIÓN
3. Revalidación	Listener en <code>visibilitychange</code> y <code>focus</code> con <code>debounce</code> de 1.2s
4. Safety timer	Si la carga supera 10s, fuerza <code>loading=false</code>
5. Nuclear Logout	Limpia claves <code>sb-*</code> en <code>localStorage/sessionStorage</code> , revoca token y redirige a /

Protección de rutas

`ProtectedRoute` en `Routes.tsx` verifica `session` y opcionalmente `allowedRoles`. Sin sesión redirige a `/login`; con rol no permitido redirige a `/`.



07 Rutas y navegación

Definidas en `src/Routes.tsx` usando **React Router v6**. Las páginas pesadas se cargan con `lazy loading` (`React.lazy`) para reducir el `bundle` inicial.

RUTA	COMPONENTE	ACCESO
/	Home	PÚBLICO
/eventos, /eventos/:id	Eventos, DetalleEvento	PÚBLICO
/lugares, /lugares/:id	Lugares, LugaresDetalle	PÚBLICO
/mapa, /contacto, /ruta/:id	MapaEventos, Contacto, RutaCompartida	PÚBLICO
/login, /registro, /reset-password	Login, Registro, ResetPassword	PÚBLICO
/perfil, /favoritos, /calendario, /rutas, /faq	MiPerfil, Favoritos, Calendario, RutaInteligente, Chat	LOGIN

RUTA	COMPONENTE	ACCESO
/dashboard, /admin/eventos, /admin/resenas	Dashboard, GestionEventos, ModeracionResenas	ADMIN/ GESTOR
/admin/usuarios	GestionUsuarios	ADMIN



08 Módulo de IA — Rutas y Chat

Generación de rutas (/rutas)

El usuario completa un wizard de 3 pasos en `RutaInteligente.tsx`. La función `geminiService.generarRuta(params)` realiza un POST a `/api/generar-ruta`, que llama a OpenRouter (Gemini 2.5 Flash primario, con fallbacks a GPT-4o-mini y Llama 3.1). El resultado es un array de paradas que se renderiza sobre un mapa Leaflet calculando la ruta a pie con la API OSRM.

CARACTERÍSTICA	DETALLE
Modelo primario	google/gemini-2.5-flash
Fallbacks	openai/gpt-4o-mini, meta-llama/llama-3.1-8b-instruct:free
Rate limit	10 peticiones/IP/hora (Upstash Redis, sliding window)
Fallback de red	Ruta estática predefinida con monumentos de Mérida si OpenRouter no es alcanzable
Reintentos cliente	Hasta 2 reintentos con backoff exponencial (1s, 2s) en errores 502/503

Chat (/faq)

`Chat.tsx` usa `geminiService.enviarMensajeStream` para enviar el mensaje e historial al endpoint `/api/chat` con `stream: true`. El servidor responde con chunks SSE que el

cliente consume en tiempo real mediante ReadableStream. En navegadores sin soporte de streaming usa fallback JSON completo. La respuesta se renderiza con ReactMarkdown.



09 API Serverless

Los endpoints residen en `/api/` y se despliegan automáticamente como **Vercel Serverless Functions**.

ENDPOINT	MÉTODO	DESCRIPCIÓN	RATE LIMIT
<code>/api/chat</code>	POST	Chat IA con streaming SSE. Body: mensaje, historial, stream.	20 msg/IP/h
<code>/api/generar-ruta</code>	POST	Genera itinerario. Body: duracion, compania, ritmo. Respuesta degradada (<code>degraded: true</code>) con ruta estática si OpenRouter no es alcanzable (ENOTFOUND).	10 req/IP/h
<code>/api/contacto</code>	POST	Envía email con datos del formulario de contacto.	—
<code>/api/eliminar-cuenta</code>	POST/DELETE	Elimina cuenta del usuario. Requiere JWT en header Authorization.	—

Servidor local de desarrollo

Para desarrollo sin Vercel CLI existe `api-server.js`, un servidor Node.js que emula las Serverless Functions en el puerto 3000. El proxy en `vite.config.ts` redirige `/api/*` hacia `http://localhost:3000`. Arrancar con `npm run dev:full` lanza Vite y el `api-server` concurrentemente.



10

Componentes principales

COMPONENTE	DESCRIPCIÓN
Navbar.tsx	Barra responsive con menú hamburguesa. Cambia estilo al hacer scroll. Muestra opciones según sesión y rol.
MapaEventos.tsx	Mapa Leaflet con marcadores, clustering en zoom bajo, panel lateral al clicar y filtros por categoría.
LazyImg.tsx	Imagen con carga diferida, efecto blur-up y fallback en error de carga.
Skeletons.tsx	Placeholders animados para el estado de carga de tarjetas de eventos, lugares y reseñas.
BotonFavorito.tsx	Botón de corazón reutilizable con optimistic UI — actualiza visualmente antes de confirmar con Supabase.
FormularioReseña.tsx	Valoración con selector de estrellas (1-5), validación y callback onReseñaPublicada.
GradientText	Texto con gradiente animado (animaciones/).
ScrollStack	Tarjetas apiladas que se despliegan con scroll — efecto stack (animaciones/).
RotatingText	Texto con rotación de palabras (animaciones/).
ScrollFloat	Texto que flota al hacer scroll (animaciones/).
FluidGlass	Forma fluida interactiva con glassmorphism (animaciones/).



11

Hooks personalizados

HOOK	ARCHIVO	RETORNO
useAuth()	AuthContext.tsx	session, profile, loading, signOut, refreshProfile

HOOK	ARCHIVO	RETORNO
<code>useFavoritos(id, tipo)</code>	<code>useFavoritos.ts</code>	<code>esFavorito</code> , <code>toggleFavorito</code> , cargando
<code>useOfflineStatus()</code>	<code>useOfflineStatus.ts</code>	<code>isOffline</code> (boolean)
<code>useSeoMeta(options)</code>	<code>useSeoMeta.ts</code>	Actualiza title, meta description y Open Graph de la página



12 PWA y Service Worker

Configurado con `vite-plugin-pwa` (Workbox) en `vite.config.ts`. El manifiesto define `display: standalone`, `theme_color: #032B43` y `start_url: /`. Los iconos PWA están en `public/pwa-192x192.png` y `public/pwa-512x512.png`.

Estrategias de caché

RECURSO	ESTRATEGIA	TTL
Assets estáticos (JS, CSS, HTML)	CacheFirst (precache)	Siempre fresco en build
Imágenes (PNG, JPG, WebP)	CacheFirst	30 días
Rutas SPA	NetworkFirst	1 día
Supabase REST (/rest/*)	StaleWhileRevalidate	24 horas
Tiles de mapa (CartoCDN)	CacheFirst	1 día
Rutas OSRM	NetworkFirst	1 hora
Leaflet (unpkg.com)	CacheFirst	30 días

El SW está configurado con un límite de 7 MiB por archivo para incluir la imagen `og-default.png` en precaché.

◆◆◆

13 Sistema de tipos TypeScript

Todas las interfaces están centralizadas en `src/types.ts`. En `src/utils/geminiService.ts` se definen los tipos específicos del módulo IA.

TIPO	ARCHIVO	DESCRIPCIÓN
Evento, CategoriaEvento	types.ts	Tabla eventos de Supabase y union type de categorías
Comentario	types.ts	Reseñas de usuarios
Perfil	types.ts	Perfil de usuario (tabla usuarios)
Favorito	types.ts	Elemento favorito del usuario
AgendaPersonal, EventoCalendario	types.ts	Eventos del calendario personal y tipo unificado
SupabaseRespuesta<T>	types.ts	Wrapper genérico de respuestas Supabase
AuthState	types.ts	Estado del contexto de autenticación
BotonFavoritoProps	types.ts	Props del componente BotonFavorito
TarjetaEventoProps	types.ts	Props del componente TarjetaEvento
FormularioReseñaProps	types.ts	Props del componente FormularioReseña
MensajeChat, ParadaRuta	geminiService.ts	Mensaje del historial y parada de itinerario IA



14

SEO y metadatos

El hook useSeoMeta se llama en cada página para establecer título, descripción, Open Graph y Twitter Card. La imagen Open Graph por defecto es /public/og-default.png. El fichero public/robots.txt controla el acceso de los crawlers. La app es una SPA con prerenderizado parcial — para SEO avanzado se recomienda considerar SSR en el futuro.



15

Consideraciones de rendimiento

TÉCNICA	DESCRIPCIÓN
Code splitting	Páginas pesadas (MapaEventos, Chat, RutaInteligente, admin) importadas con React.lazy()
Chunks de vendor	Separación en vendor-react, vendor-leaflet, vendor-supabase para caché independiente
Lazy images	Componente LazyImg con loading="lazy" y efecto blur-up
Optimización de imágenes	Se recomienda convertir /public/Imagenes/ a WebP para reducir peso
Bundle analysis	npm run build -- --report genera informe de chunks
Límite SW	Service Worker configurado con límite de 7 MiB por archivo en precaché